Лабораторная работа №11
Тестирование производительности ПО

1 Цель работы

1.1 Освоить процесс профилирования ПО

1.2 Изучить методы тестирования производительности ПО

2 Литература

2.1 Игнатьев, А. В. Тестирование программного обеспечения : учебное пособие для вузов / А. В. Игнатьев. — 4-е изд., стер. — Санкт-Петербург : Лань, 2025. — 56 с. — ISBN 978-5-507-50858-7. — Текст : электронный // Лань : электронно-библиотечная система. — URL: https://e.lanbook.com/book/481331 (дата обращения: 21.05.2025). — Режим доступа: для авториз. пользователей.

3 Подготовка к работе

3.1 Повторить теоретический материал (см. п.2).

3.2 Изучить описание лабораторной работы.

4 Основное оборудование

4.1 Персональный компьютер.

5 Задание

5.1 Тестирование производительности методов при помощи Benchmark

5.1.1 Создать новый проект консольного приложения

5.1.2 Установить Nuget-пакет BenchmarkDotNet

5.1.3 Создать новый публичный класс PrimesBenchmark с атрибутом [MemoryDiagnoser], в него добавить 2 статических метода:

```
private static bool IsPrimeNaive(int n)
{
    for (int i = 2; i < n; i++)
        if (n % i == 0) return false;
    return true;
}

private static bool IsPrimeFast(int n)
{
    if (n < 2) return false;
    if (n == 2) return true;
    if (n % 2 == 0) return false;
    int sqrt = (int)Math.Sqrt(n);
    for (int i = 3; i <= sqrt; i += 2)
        if (n % i == 0) return false;
    return true;
}
```

5.1.4 Добавить в класс публичное целочисленное поле N. Укажите для него атрибут [Params(10_000, 50_000, 100_000)] для тестирования работы методов на различных входных данных

5.1.5 Добавить в класс 2 публичных метода без параметров – один

вычисляет сумму простых чисел от 2 до n при помощи метода IsPrimeNaive, второй – при помощи метода IsPrimeFast.

5.1.6 Для методов указать атрибут [Benchmark], причем для первого указать параметр атрибута Baseline = true, чтобы производительность вычислялась относительно него.

5.1.7 Изменить конфигурацию сборки с Debug на Release

5.1.8 Запустить бенчмарк в основной программе при помощи BenchmarkRunner.Run<PrimesBenchmark>();

5.1.9 Дождаться завершения бенчмарка, зафиксировать полученные показатели в **отчет**.

5.1.10 Сравнить полученные показатели для двух методов на различных наборах данных. Сделать вывод на основе полученных результатов и записать в **отчет**.

5.2 Профилирование производительности методов

5.2.1 Создать новый проект консольного приложения

```
class Program
{
    static void Main()
    {
        const string filePath = "server.log";
        if (!File.Exists(filePath))
            File.WriteAllLines(filePath,
GenerateFakeLogs(10_000_000));

        var sw = Stopwatch.StartNew();
        var result = AnalyzeLogs(filePath);
        sw.Stop();

        Console.WriteLine($"Ошибок:          {result.errorCount},
предупреждений: {result.warningCount}");
        Console.WriteLine($"Время                    выполнения:
{sw.ElapsedMilliseconds} мс");
    }


    static (int errorCount, int warningCount) AnalyzeLogs(string
path)
    {
        var lines = File.ReadAllLines(path);

        int errorCount = lines.Count(l => l.Contains("ERROR"));
        int      warningCount      =      lines.Count(l      =>
l.Contains("WARNING"));

        return (errorCount, warningCount);
    }

    static IEnumerable<string> GenerateFakeLogs(int count)
    {
        var rand = new Random();
```

```
        string[] types = { "INFO", "WARNING", "ERROR" };
        for (int i = 0; i < count; i++)
        {
            yield return $"{DateTime.Now:HH:mm:ss}
[{types[rand.Next(types.Length)]}] Message {i}";
        }
    }
}
```

5.2.2 Изменить конфигурацию сборки с Debug на Release

5.2.3 При помощи меню Отладка – Профилировщик производительности запустить отладку с опцией «Использование ЦП»

5.2.4 Оценить время выполнения кода, затраченные ресурсы, определить критический участок в программе. Выявить, каким образом этот участок сказывается на производительности программы. Зафиксировать выводы в **отчет**.

5.2.5 Оптимизировать работу метода и выполнить профилирование повторно, сравнить результаты профилирования до и после оптимизации, зафиксировать выводы в **отчет**.

5.3 Нагрузочное тестирование при помощи k6

5.3.1 Создать новый проект minimal-api

```
using System.Security.Cryptography;

var builder = WebApplication.CreateBuilder(args);
builder.Logging.SetMinimumLevel(LogLevel.Warning);
builder.WebHost.ConfigureKestrel(options =>
options.Limits.MaxRequestBodySize = 4096);
var app = builder.Build();

app.MapPost("/password-hashes", (HashRequest request) =>
{
    if (string.IsNullOrWhiteSpace(request.Password) ||
request.Password.Length > 256)
        return Results.BadRequest(new { error = "Password must
contain 1 to 256 characters and not be whitespace." });

    const int iterations = 600_000;
    var salt = RandomNumberGenerator.GetBytes(16);
    var hash = Rfc2898DeriveBytes.Pbkdf2(
        request.Password, salt, iterations,
HashAlgorithmName.SHA256, 32);

    return Results.Ok(new
    {
        algorithm = "PBKDF2-SHA256",
        iterations,
        salt = Convert.ToBase64String(salt),
        hash = Convert.ToBase64String(hash)
    });
});

app.Run("http://localhost:5080");
```

```
record HashRequest(string? Password);
```

5.3.2 Создайте новый сценарий k6 для проведения стрессового тестирования API с увеличением нагрузки. Необходимо оценить предельно допустимую нагрузку на API.

Необходимо последовательно запускать k6 с n пользователями в течение 15 секунд. После каждого запуска увеличивайте значение n в 2 раза. При обнаружении значения n при котором тесты не проходят, повторно запустите неуспешный набор и последний успешный наборы для закрепления результатов. Зафиксируйте результаты в **отчет**

Указать следующие критерии для провала теста:

```
thresholds: {
    http_req_failed: ['rate<0.01'],
    http_req_duration: ['p(95)<500'],
},
```

Код для отправки запроса:

```
const url = 'http://localhost:5080/password-hashes';
const body = JSON.stringify({ password: 'LoadTest-Only-2026!' });
const response = http.post(url, body, {
    headers: { 'Content-Type': 'application/json' },
    timeout: '2s',
});
```

6 Порядок выполнения работы

6.1 Повторить теоретический материал п. 3.1;
6.2 Выполнить задания п.5
6.3 Ответить на контрольные вопросы п.8;
6.4 Заполнить отчет п. 7.

7 Содержание отчета

7.1 Титульный лист;
7.2 Цель работы;
7.3 Показатели, полученные в бенчмарке и вывод на их основе;
7.4 Описание критического участка в коде и его влияния на программу;
7.5 Результаты профилирования;
7.6 Результаты тестирования k6;
7.7 Ответы на контрольные вопросы п. 6.3;
7.8 Вывод по проделанной работе.

8 Контрольные вопросы

8.1 Как запустить профилировщик производительности в Visual Studio?
8.2 Как просмотреть использование ЦП в профилировщике?
8.3 Как использовать BenchmarkDotNet?
8.4 Что такое нагрузочное тестирование?
8.5 Что такое стрессовое тестирование?